Reviewer Pack — Lie Stabilizer Dimension Linearly Bounds Arithmetic Formula …

Entry 44fe923c126a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 02:17:54 UTC

Lie Stabilizer Dimension Linearly Bounds Arithmetic Formula Size

Entry ID: `44fe923c126a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 02:17:54 UTC

1. Conjecture

Field A (mathematical branch): Geometric Invariant Theory (sl_N Lie-algebra stabilizers of homogeneous forms) **Field B** (complexity object): Arithmetic formula size for homogeneous polynomials (GCT-style hardness for det vs perm)

Statement:

> Let f ∈ Q[x_1,…,x_N] be a homogeneous polynomial of degree d ≥ 3 that uses every variable (∂f/∂x_i ≠ 0 ∀ i), and define its reduced Lie stabilizer rLS(f) := dim { A ∈ sl_N : Σ_{i,j} A_{ij} x_j ∂f/∂x_i ≡ 0 in Q[x] }. We conjecture that for every such f computable by an arithmetic formula F of size s (counted as the number of internal +/× gates), rLS(f) ≤ 4·s. A single (formula F, expanded polynomial f, computed rLS) triple with rLS(f) > 4·s refutes the conjecture; conversely, since rLS(det_n) = 2n²−2, any formula for det_n has size ≥ (n²−1)/2.

Rationale (proposer's reasoning):

> Mulmuley–Sohoni's program isolates polynomials whose stabilizer groups are large (det_n is 'characterized by its stabilizer') and predicts their circuit-rigidity reflects that algebraic symmetry, but no explicit numeric bridge from stabilizer dimension to formula size has been proven or empirically pinned down. A linear inequality rLS ≤ 4s would give a missing GCT sub-lemma that is computable, sidesteps the natural-proofs barrier (rLS is not large for random truth tables of Boolean functions and is defined on algebraic, not Boolean, objects), and dodges algebrization (it is a property of f as a point in P(Sym^d), not of an oracle access pattern). It targets the permanent-vs-determinant axis directly through the Lie-algebra side of stabilizer theory.

Taxonomy category: `GCT` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `713002a6ac835823`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 840 trials (30 seeds × 7 sizes s∈{3,5,8,12,16,24,32} × 4 (N,d) settings), compute rLS−4s per trial. Conjecture is SUPPORTED iff max(rLS−4s) ≤ 0 over all trials AND the rLS routine reproduces rLS(det_2)=7, rLS(det_3)=16, rLS(perm_3)=8 exactly.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): `- Lie algebra stabilizer homogeneous polynomial arithmetic formula lower bound` `- geometric complexity theory symmetry determinant permanent formula size` `- Lie stabilizer dimension polynomial GCT orbit closure lower bounds`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def extended_gcd(a, b):
    if a == 0:
        return (b, 0, 1)
    else:
        g, y, x = extended_gcd(b % a, a)
        return (g, x - (b // a) * y, y)

def mod_inverse(a, m):
    g, x, _ = extended_gcd(a, m)
    if g != 1:
        raise ValueError('Modular inverse does not exist')
    else:
        return x % m
```

```python
def matrix_multiply(A, B, mod):
    n = len(A)
    C = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % mod
    return C

def matrix_rank(A, mod):
    n = len(A)
    rank = 0
    for i in range(n):
        if A[i][i]:
            for j in range(i+1, n):
                factor = (A[j][i] * mod_inverse(A[i][i], mod)) % mod
                for k in range(n):
                    A[j][k] = (A[j][k] - factor * A[i][k]) % mod
            rank += 1
    return rank

def generate_polynomial(N, d, variables):
    if d == 0:
        return {(): Fraction(1)}
    elif d == 1:
        return {(var,): random.choice([Fraction(-1), Fraction(1)]) for var in variables}
    else:
        result = {}
        for i in range(len(variables)):
            new_vars = variables[:i] + variables[i+1:]
            for term, coeff in generate_polynomial(N, d-1, new_vars).items():
                result[(variables[i],) + term] = coeff * random.choice([Fraction(-1), Fraction(1)]
        return result

def expand_polynomial(poly):
    n = len(poly)
    expanded = {}
    for term, coeff in poly.items():
        monomial = 1
        for var, exp in term:
            monomial *= var**exp
        expanded[monomial] = coeff
    return expanded

def compute_rLS(f, N):
    x = [Fraction(0)] * N
    df = [(sum(coeff * var**(exp-1) for var, exp in term.items()) for term, coeff in f.items()) fo
    A = [[0] * (N**2) for _ in range(N)]
    for i in range(N):
        for j in range(N):
            for k in range(N):
                A[i][j*N + k] = df[j][i] * x[k]
    return N**2 - 1 - matrix_rank(A, mod=10007)

def generate_formulas(N, d, s):
```

```
    variables = [Fraction(1)] * N
    formulas = []
    for _ in range(s):
        formula = generate_polynomial(N, d, variables)
        if all(coeff != Fraction(0) for coeff in formula.values()):
            formulas.append(formula)
    return formulas

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for N, d in [(4, 3), (6, 3), (6, 4), (9, 3)]:
        for s in [3, 5, 8, 12, 16, 24, 32]:
            formulas = generate_formulas(N, d, s)
            for formula in formulas:
                f = expand_polynomial(formula)
                rLS = compute_rLS(f, N)
                results.append((s, rLS))
    max_diff = max(rLS - 4 * s for s, rLS in results)
    conjecture_holds = max_diff <= 0
    counterexample = "" if conjecture_holds else f"max_diff={max_diff}"
    return {
        "metric_name": "rLS - 4s",
        "metric_value": max_diff,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3**j + 5**k for i in range(5) for j in range
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r for r in results if not r["conjecture_holds"])["seed"]
        print(f"RESULT: FALSIFIED counterexample='max_diff={max(r['metric_value'] - 4 * s for s, r
    else:
        print("RESULT: INCONCLUSIVE insufficient data to make a definitive conclusion")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5b89c1e8.py", line 129, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5b89c1e8.py", line 110, in run_trial
    f = expand_polynomial(formula)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5b89c1e8.py", line 79, in expand_polyno
    for var, exp in term:
        ^^^^^^^^
TypeError: cannot unpack non-iterable Fraction object
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed with a TypeError in expand_polynomial before any of the 840 trials or calibration checks could run, so neither the support nor falsification condition can be evaluated. | next: Fix the polynomial expansion routine (term iteration unpacks a Fraction) so monomials are represented as (coefficient, [(var, exp), ...]) tuples, then rerun the 840-trial sweep plus the det_2/det_3/perm_3 calibration.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	180337
2	preregistration	claude_max	opus	0	0	6082
3	novelty	claude_max	opus	0	0	8318
4	novelty	claude_max	opus	0	0	7695
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22126
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15338
7	judge	claude_max	opus	0	0	7474

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 247369 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/44fe923c126a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/44fe923c126a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/44fe923c126a.tar.gz` (if generated)